

Curso Prático de JavaScript

Do Básico à Manipulação de Páginas Web

Estrutura do Curso

- Ensino Modular e Progressivo
- Desafios Práticos ao Fim de Cada Módulo
- Resoluções Passo a Passo

Conteúdo

1	Módulo 1: Fundamentos Básicos	2
1.1	Variáveis e Constantes	2
1.2	Tipos de Dados	2
2	Módulo 2: Estruturas de Controle	3
2.1	Resolução do Exercício do Módulo 1	3
2.2	Condicionais (If/Else)	3
2.3	Laços de Repetição (Loops)	3
3	Módulo 3: Funções	4
3.1	Resolução do Exercício do Módulo 2	4
3.2	O que são Funções?	4
3.3	Retorno de Funções	4
4	Módulo 4: Arrays e Objetos	5
4.1	Resolução do Exercício do Módulo 3	5
4.2	Arrays (Listas)	5
4.3	Objetos Literais	5
5	Módulo 5: Manipulação do DOM (Páginas Web)	6
5.1	Resolução do Exercício do Módulo 4	6
5.2	O que é o DOM?	6
5.3	Selecionando e Alterando Elementos	6
5.4	Eventos (Clicks)	6
6	Módulo 6: JavaScript Moderno (ES6+)	7
6.1	Resolução do Exercício do Módulo 5	7
6.2	Arrow Functions	7
6.3	Desestruturação (Destructuring)	7
7	Módulo 7: Assincronismo e Promises	8
7.1	Resolução do Exercício do Módulo 6	8
7.2	O que é Assincronismo?	8
7.3	Entendendo Promises	8
8	Módulo 8: Async/Await e Consumo de APIs (Fetch)	9
8.1	Resolução do Exercício do Módulo 7	9
8.2	Async / Await	9
8.3	Consumindo APIs com Fetch	9
9	Módulo 9: Programação Orientada a Objetos (POO)	10
9.1	Resolução do Exercício do Módulo 8	10
9.2	Classes e Objetos	10

1 Módulo 1: Fundamentos Básicos

Bem-vindo ao mundo do JavaScript! Neste primeiro módulo, vamos entender como a linguagem armazena e lida com informações na memória.

1.1 Variáveis e Constantes

Em JavaScript, usamos variáveis para guardar dados. Hoje em dia, utilizamos `let` e `const` (evite usar o antigo `var`).

- **let**: Usado para variáveis cujo valor pode mudar durante a execução do programa.
- **const**: Usado para constantes, ou seja, valores que **não** podem ser reatribuídos depois de criados.

```
let idade = 25;
idade = 26; // Isso é permitido

const nome = "Maria";
// nome = "João"; // Erro! Constantes não podem ser alteradas.
```

1.2 Tipos de Dados

O JavaScript possui vários tipos de dados, sendo os mais comuns:

- **String**: Texto, sempre entre aspas simples (") ou duplas (").
- **Number**: Números inteiros ou decimais (ex: 10, 3.14).
- **Boolean**: Verdadeiro (`true`) ou Falso (`false`).

Exercício Prático 1

Desafio: Crie um script simulando o cadastro de um produto. Você deve declarar: 1. Uma constante para o **nome** do produto. 2. Uma variável para o **preço** do produto. 3. Uma variável booleana indicando se o produto **está em estoque**. Ao final, exiba essas informações no console usando `console.log()`.

2 Módulo 2: Estruturas de Controle

2.1 Resolução do Exercício do Módulo 1

Antes de avançarmos, veja como fica a resolução do desafio anterior:

```
// Resolução - Módulo 1
const nomeProduto = "Notebook M1";
let preco = 5500.99;
let emEstoque = true;

console.log("Produto: " + nomeProduto);
console.log("Preço: R$ " + preco);
console.log("Disponível: " + emEstoque);
```

2.2 Condicionais (If/Else)

As condicionais permitem que o código tome decisões baseado em certas condições.

```
let nota = 7.5;

if (nota >= 7) {
  console.log("Aprovado!");
} else {
  console.log("Reprovado.");
}
```

2.3 Laços de Repetição (Loops)

Para repetir uma ação várias vezes, usamos os "loops", como o `for`. A estrutura do `for` recebe três partes: a inicialização (`let i = 0`), a condição de parada (`i < 5`), e o incremento (`i++`).

```
// Vai imprimir de 0 até 4 no console
for (let i = 0; i < 5; i++) {
  console.log("Contagem: " + i);
}
```

Exercício Prático 2

Desafio: Crie um laço de repetição (loop `for`) que conte de 0 até 20. Porém, dentro desse loop, use uma condicional (`if`) para imprimir no `console.log()` **apenas os números pares**.

3 Módulo 3: Funções

3.1 Resolução do Exercício do Módulo 2

Aqui está a lógica para descobrir os números pares em um loop usando o operador módulo (%), que pega o resto da divisão.

```
// Resolução - Módulo 2
for (let i = 0; i <= 20; i++) {
  // Se o resto da divisão por 2 for 0, é par
  if (i % 2 === 0) {
    console.log("Número par encontrado: " + i);
  }
}
```

3.2 O que são Funções?

Funções são blocos de código reutilizáveis projetados para realizar uma tarefa específica. Você as declara uma vez e as utiliza (chama) quantas vezes quiser.

```
// Declaração da função
function saudarUsuario(nome) {
  console.log("Olá, " + nome + "! Bem-vindo.");
}

// Chamando a função
saudarUsuario("Carlos");
saudarUsuario("Ana");
```

3.3 Retorno de Funções

Muitas vezes, queremos que a função processe um dado e nos devolva o resultado em vez de apenas imprimir. Para isso, usamos a palavra-chave `return`.

```
function multiplicar(a, b) {
  return a * b;
}

let resultado = multiplicar(5, 4);
console.log(resultado); // Imprime 20
```

Exercício Prático 3

Desafio: Crie uma função chamada `verificarMaioridade`. Ela deve receber um parâmetro chamado `idade`. A função deve verificar: se a idade for maior ou igual a 18, **retornar** a string "Maior de idade". Caso contrário, **retornar** "Menor de idade". Ao final, chame a função passando sua idade e imprima o retorno no console.

4 Módulo 4: Arrays e Objetos

4.1 Resolução do Exercício do Módulo 3

```
// Resolução - Módulo 3
function verificarMaioridade(idade) {
  if (idade >= 18) {
    return "Maior de idade";
  } else {
    return "Menor de idade";
  }
}

let status = verificarMaioridade(20);
console.log(status); // Imprime "Maior de idade"
```

4.2 Arrays (Listas)

Arrays são usados para guardar múltiplos valores em uma única variável. Você acessa os itens através do seu índice, que começa em zero (0).

```
let frutas = ["Maçã", "Banana", "Laranja"];
console.log(frutas[0]); // Imprime "Maçã"

// Adicionando um item no final
frutas.push("Uva");
```

4.3 Objetos Literais

Objetos são coleções de pares "chave/valor", ideais para representar entidades complexas do mundo real.

```
let carro = {
  marca: "Ford",
  modelo: "Mustang",
  ano: 1969
};

console.log(carro.marca); // Imprime "Ford"
```

Exercício Prático 4

Desafio: Crie um objeto representando você, contendo as propriedades `nome`, `idade` e `cidade`. Em seguida, crie uma frase (string) acessando as propriedades do seu objeto e exiba no console. (Exemplo: "Olá, meu nome é João, tenho 25 anos e moro em São Paulo.")

5 Módulo 5: Manipulação do DOM (Páginas Web)

5.1 Resolução do Exercício do Módulo 4

```
// Resolução - Módulo 4
let pessoa = {
  nome: "Lucas",
  idade: 30,
  cidade: "Rio de Janeiro"
};

console.log("Olá, meu nome é " + pessoa.nome +
  ", tenho " + pessoa.idade +
  " anos e moro em " + pessoa.cidade + ".");
```

5.2 O que é o DOM?

O **DOM** (Document Object Model) é a representação que o navegador faz do HTML da sua página. Com JavaScript, podemos alterar o HTML, o CSS e reagir a eventos (como cliques) dinamicamente.

5.3 Selecionando e Alterando Elementos

Podemos buscar um elemento no HTML pelo seu id usando `document.getElementById()`.

```
// HTML: <p id="meuParagrafo">Texto Original</p>

// Selecionando no JavaScript
let paragrafo = document.getElementById("meuParagrafo");

// Alterando o texto
paragrafo.innerText = "Texto alterado com JavaScript!";
```

5.4 Eventos (Clicks)

```
let botao = document.getElementById("meuBotao");

botao.addEventListener("click", function() {
  alert("O botão foi clicado!");
});
```

Exercício Prático 5

Desafio Prático: Imagine que você tem no seu HTML um `<h1 id="titulo">Olá Mundo</h1>` e um `<button id="btnMudar">Mudar Título</button>`. Escreva o código JavaScript necessário para que, ao clicar no botão, o texto do `h1` mude para "JavaScript é incrível!".

6 Módulo 6: JavaScript Moderno (ES6+)

6.1 Resolução do Exercício do Módulo 5

```
// Resolução - Módulo 5
let titulo = document.getElementById("titulo");
let btnMudar = document.getElementById("btnMudar");

btnMudar.addEventListener("click", function() {
    titulo.innerText = "JavaScript é incrível!";
});
```

6.2 Arrow Functions

As *Arrow Functions* (funções de seta) foram introduzidas no ES6 e oferecem uma sintaxe mais curta e limpa para escrever funções, além de possuírem comportamentos específicos com a palavra-chave `this`.

```
// Função tradicional
function somar(a, b) {
    return a + b;
}

// Arrow Function equivalente
const somarArrow = (a, b) => a + b;
```

6.3 Desestruturação (Destructuring)

A desestruturação permite extrair dados de arrays ou propriedades de objetos em variáveis distintas de forma muito rápida.

```
const usuario = { nome: "Ana", idade: 28, cargo: "Desenvolvedora" };

// Extraíndo propriedades diretamente para variáveis
const { nome, cargo } = usuario;

console.log(nome); // Imprime "Ana"
console.log(cargo); // Imprime "Desenvolvedora"
```

Exercício Prático 6

Desafio: Converta a seguinte função para uma *Arrow Function* de linha única e use *desestruturação* para extrair a propriedade `preco` do objeto passado como parâmetro.

```
function exibirPreco(produto) { return "O preço é " + produto.preco; }
```

7 Módulo 7: Assincronismo e Promises

7.1 Resolução do Exercício do Módulo 6

```
// Resolução - Módulo 6
// Usando arrow function e desestruturando ({ preco }) direto no parâmetro
const exibirPreco = ({ preco }) => "O preço é " + preco;

const meuProduto = { nome: "Teclado", preco: 250 };
console.log(exibirPreco(meuProduto)); // Imprime "O preço é 250"
```

7.2 O que é Assincronismo?

Normalmente, o JavaScript executa o código de cima para baixo de forma "síncrona". Porém, algumas tarefas demoram a ser concluídas (como baixar uma imagem ou buscar dados de um servidor). O assincronismo permite que o programa continue executando outras tarefas enquanto espera essas operações lentas terminarem.

7.3 Entendendo Promises

Uma Promise (promessa) é um objeto que representa a eventual conclusão (ou falha) de uma operação assíncrona. Ela pode estar "pendente", "resolvida" (sucesso) ou "rejeitada" (erro).

```
const promessa = new Promise((resolve, reject) => {
  let sucesso = true; // Simulando um cenário

  if (sucesso) {
    resolve("Operação concluída com sucesso!");
  } else {
    reject("Ocorreu um erro no processo.");
  }
});

// Consumindo a promise
promessa
  .then(resultado => console.log(resultado)) // Captura o resolve
  .catch(erro => console.log(erro));         // Captura o reject
```

Exercício Prático 7

Desafio: Crie uma Promise que simula um atraso usando a função `setTimeout`. A Promise deve ser resolvida após 2 segundos (2000 milissegundos) retornando a string "Dados carregados!". Consuma essa Promise usando `.then()` para imprimir a mensagem no console.

8 Módulo 8: Async/Await e Consumo de APIs (Fetch)

8.1 Resolução do Exercício do Módulo 7

```
// Resolução - Módulo 7
const simularAtraso = new Promise((resolve) => {
  setTimeout(() => {
    resolve("Dados carregados!");
  }, 2000);
});

simularAtraso.then((mensagem) => {
  console.log(mensagem);
});
```

8.2 Async / Await

Introduzidas em versões mais recentes do JavaScript, as palavras `async` e `await` são uma forma mais elegante e legível de lidar com Promises, fazendo o código assíncrono parecer síncrono.

```
async function executarTarefa() {
  try {
    // Pausa a execução da função até a promise resolver
    const mensagem = await simularAtraso;
    console.log(mensagem);
  } catch (erro) {
    console.log(erro);
  }
}
```

8.3 Consumindo APIs com Fetch

A função global `fetch` é utilizada para fazer requisições HTTP e buscar dados de outros servidores na internet (APIs). O `fetch` retorna uma Promise.

```
async function buscarUsuario() {
  // Busca dados em formato JSON de uma API pública
  const resposta = await fetch('https://jsonplaceholder.typicode.com/users/1');
  const usuario = await resposta.json(); // Converte a resposta para objeto JavaScript

  console.log(usuario.name);
}

buscarUsuario();
```

Exercício Prático 8

Desafio: Crie uma função assíncrona (`async`) que faça uma requisição com `fetch` para a API pública do Pokémon: <https://pokeapi.co/api/v2/pokemon/pikachu>. Converta a resposta para JSON e imprima no console o nome do Pokémon (propriedade `name`).

9 Módulo 9: Programação Orientada a Objetos (POO)

9.1 Resolução do Exercício do Módulo 8

```
// Resolução - Módulo 8
async function buscarPikachu() {
  try {
    let resposta = await fetch("https://pokeapi.co/api/v2/pokemon/pikachu");
    let dados = await resposta.json();
    console.log("Nome do Pokémon encontrado: " + dados.name);
  } catch (erro) {
    console.log("Erro na requisição:", erro);
  }
}

buscarPikachu();
```

9.2 Classes e Objetos

A Programação Orientada a Objetos permite organizar o código simulando coisas do mundo real. O JavaScript utiliza **Classes** como se fossem “plantas” ou “moldes” para fabricar objetos.

```
class Pessoa {
  // O construtor é executado sempre que criamos uma NOVA Pessoa
  constructor(nome, idade) {
    this.nome = nome;
    this.idade = idade;
  }

  // Comportamentos (Métodos) da classe
  seApresentar() {
    console.log(`Olá, eu sou ${this.nome} e tenho ${this.idade} anos.`);
  }
}

// Instanciando (criando) novos objetos a partir do molde
const pessoa1 = new Pessoa("Pedro", 22);
const pessoa2 = new Pessoa("Mariana", 29);

pessoa1.seApresentar(); // Imprime a frase com os dados de Pedro
```

Exercício Prático Final (Módulo 9)

Desafio Prático Avançado: Crie uma classe `ContaBancaria` que tenha as propriedades `titular` (texto) e `saldo` (iniciando sempre em 0 no construtor). Adicione dois métodos a ela: `depositar(valor)` e `sacar(valor)`. **Regra de negócio:** O método `sacar` só deve subtrair o valor se houver saldo suficiente na conta; caso contrário, deve exibir um `console.log` de “Saldo insuficiente”.

Por fim, instancie uma conta no seu nome, deposite 100 reais, tente sacar 150 reais, depois saque 50 reais e imprima o saldo final.

Fim do Curso Avançado!

Parabéns por concluir esta jornada! Você evoluiu das variáveis e loops até dominar requisições assíncronas em APIs e Orientação a Objetos no JavaScript.